

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

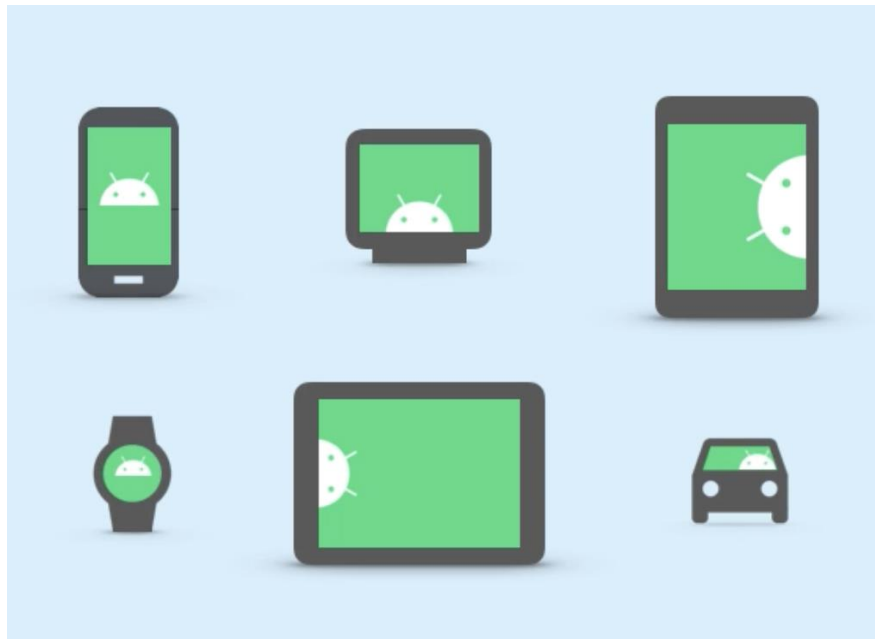
Layouts: diseñando interfaces



Universidad
Rey Juan Carlos

Diseño de interfaces en Android

- Los dispositivos Android vienen en muchos factores de forma diferentes.
- Cada vez hay más píxeles por pulgada en las pantallas de los dispositivos.
- Los desarrolladores necesitan poder especificar dimensiones de diseño que sean coherentes en todos los dispositivos.

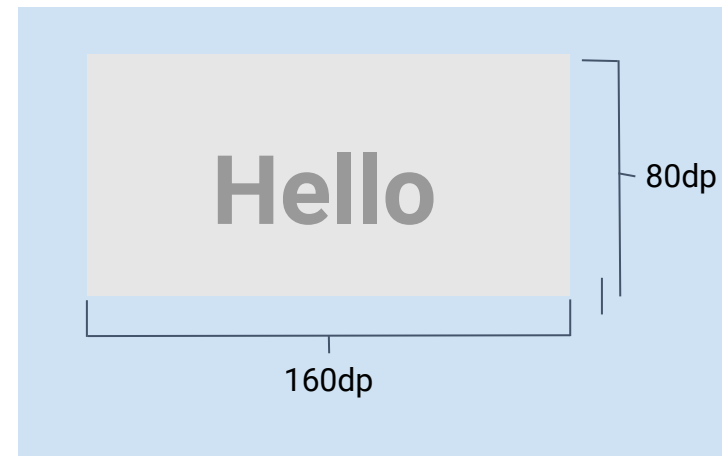


Píxeles independientes de la densidad (dp)

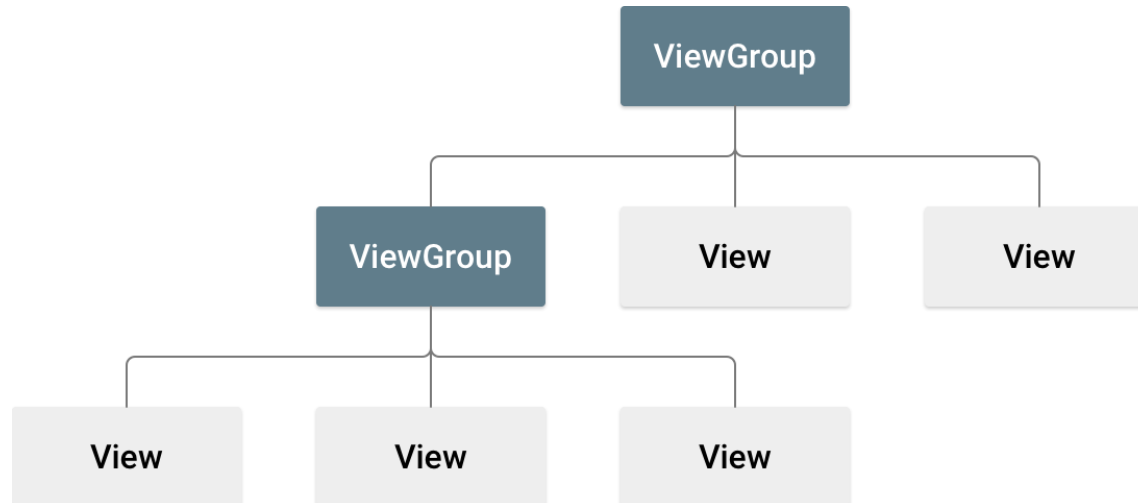
Utilice dp cuando especifique tamaños en su maqueta, como la anchura o la altura de las vistas.

- Los píxeles independientes de la densidad (dp) tienen en cuenta la densidad de la pantalla.
- Las vistas de Android se miden en píxeles independientes de la densidad.

Density qualifier	DPI estimate
ldpi (mostly unused)	~120dpi
mdpi (baseline density)	~160dpi
hdpi	~240dpi
xhdpi	~320dpi
xxhdpi	~480dpi
xxxhdpi	~640dpi



Organización de un layout



View: se suelen llamar widgets, y muestran los objetos que el usuario ve en pantalla. Ejemplos: Button, Textview.

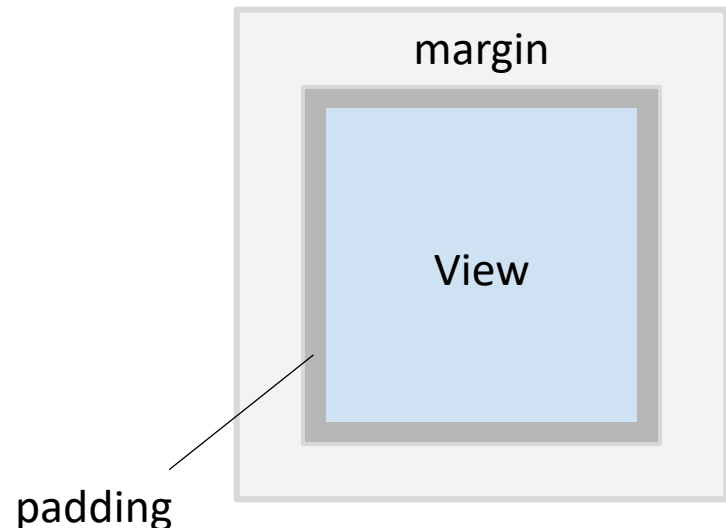
ViewGroup: conocidos como diseños, definen estructuras o contenedores en los que organizar elementos *View* y otros *ViewGroup*. Ejemplos: ConstraintLayout

Se pueden declarar en los archivos de layout XML, o crear en tiempo de ejecución.

Parámetros de diseño más comunes

- **id**: identificador único del elemento en la interfaz. Las referencias a la ID de un elemento se usan para asociarlo a los objetos declarados en las Activities (usando *findViewById*), así como al definir las relaciones con otros objetos de la interfaz.
- **layout_width/layout_height**: definen el tamaño del elemento con medidas exactas (no recomendable) o las constantes *wrap_content* y *match_parent*.
- **layout_something**: otros parámetros de diseño que definen márgenes, padding, o relaciones de posición entre elementos.

```
<Button  
    android:id="@+id/my_button"  
    android:layout_width="wrap_content"  
    android:layout_height="match_parent"  
    android:text="@string/my_button_text" />
```

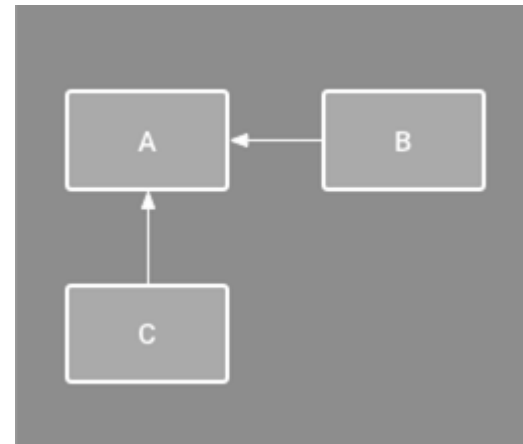


ConstraintLayout

ConstraintLayout

Es un *ViewGroup* que organiza los elementos hijos en posiciones relativas. Dicha posición puede definirse mediante restricciones con elementos del mismo nivel, o respecto a la vista padre.

El uso de *ConstraintLayout* permite diseñar interfaces de manera simplificada manteniendo una jerarquía más plana, evitando anidar *ViewGroups* y, por tanto, mejorando el rendimiento.



Posicionamiento en un ConstraintLayout

layout_constraint<Origen>_to<Destino>: parent | viewID

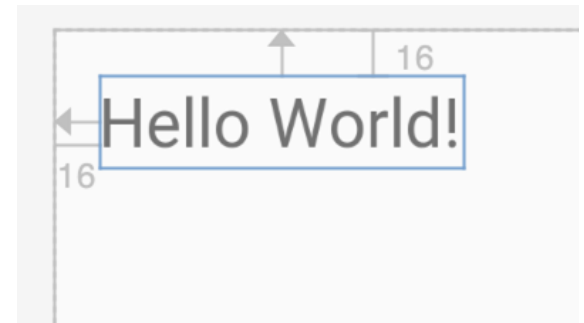
Establece el alineamiento del lado <Origen> del elemento que tiene el atributo respecto al lado Destino del elemento relativo sobre el que se posiciona.

- Origen | Destino = *Start* (izq), *End* (dcha), *Top* (arriba) y *Bottom* (abajo)

Ejemplo de atributos en un TextView:

```
app:layout_constraintTop_toTopOf="parent"
```

```
app:layout_constraintLeft_toLeftOf="parent"
```

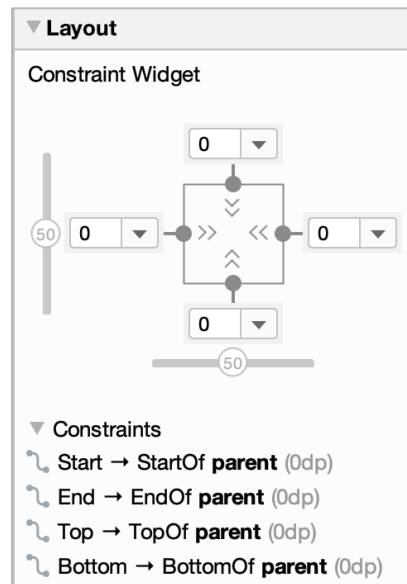


parent hace referencia al contenedor (ViewGroup) que engloba a un elemento. En este caso, al *ConstraintLayout*.

Tipos de restricciones

Usando el Layout Editor junto al Constraint Widget del panel de atributos (a la derecha) se puede ajustar el posicionamiento de un elemento de forma intuitiva.

La vista respecto a la que se posiciona el elemento seleccionado se puede ajustar con el ratón desde el Layout Editor. Hay tres tipos de restricción:



Fixed



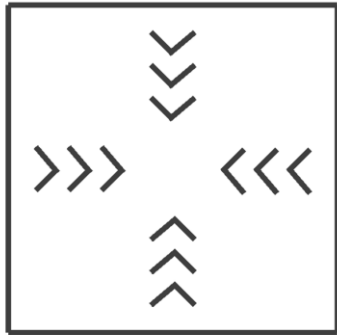
Wrap content



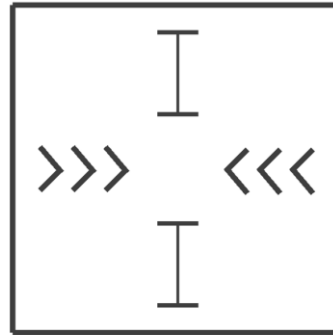
Match constraints

Tipos de restricciones

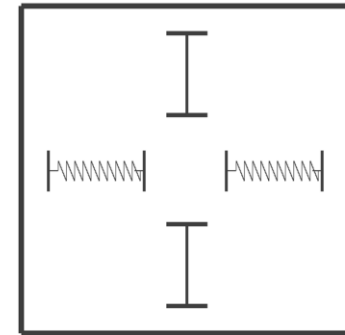
El ancho y alto de los elementos se puede ajustar para que se ajuste al contenido, tenga un valor fijo, o se adapte para cumplir las restricciones de la vista.



layout_width `wrap_content`
layout_height `wrap_content`

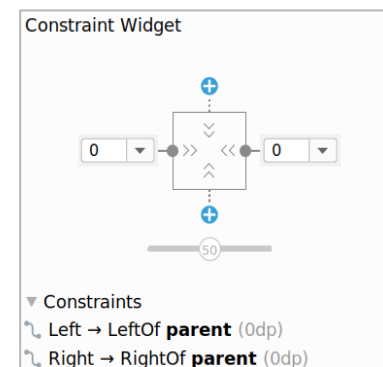


layout_width `wrap_content`
layout_height `48dp`



layout_width `0dp(match_constraint)`
layout_height `48dp`

Si las restricciones opuestas están fijadas y no se les asigna una distancia (0dp), el elemento se centra.



Ejemplo de ConstraintLayout

activity_main.xml

```
<androidx.constraintlayout.widget.ConstraintLayout
```

```
...
```

```
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    tools:context=".MainActivity">
```

```
...
```

```
<TextView
```

```
...
```

```
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    app:layout_constraintBottom_toBottomOf="parent"  
    app:layout_constraintEnd_toEndOf="parent"  
    app:layout_constraintStart_toStartOf="parent"  
    app:layout_constraintTop_toBottomOf="@+id/button"/>
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```

Ajuste relativo a la vista padre
a la izquierda, arriba y abajo

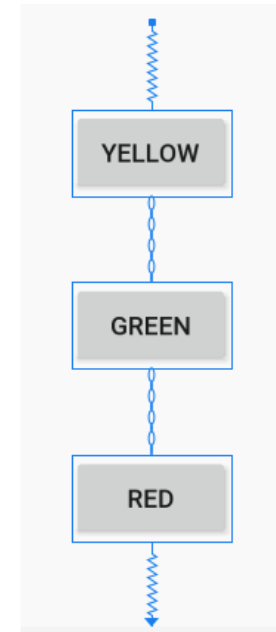
Ajuste relativo a
otro elemento
dentro del
ConstraintLayout

Cadenas de elementos

- Permiten posicionar unos elementos respecto a otros
- Pueden enlazarse vertical u horizontalmente
- Proporcionan la funcionalidad anteriormente provista por *LinearLayout*

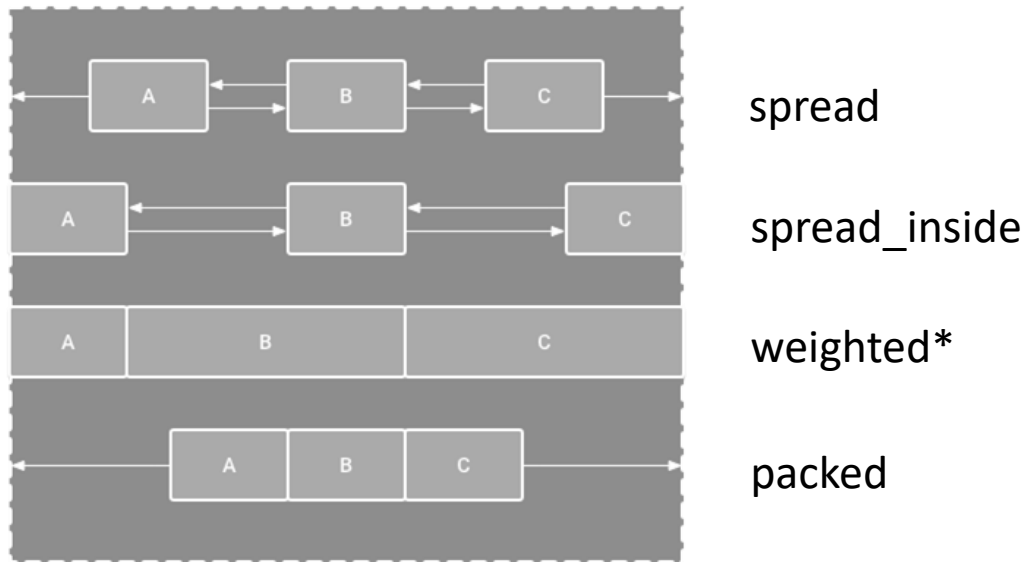
Creación de una cadena desde el Layout Editor

1. Seleccionar los elementos que formen la cadena
2. Hacer clic con el botón derecho y seleccionar *Chains*
3. Crear una cadena horizontal o vertical



Tipos de cadenas de elementos

Hay diversos criterios para ajustar el espacio entre las vistas encadenadas:



Se debe añadir a la primera View que forma la cadena el siguiente atributo:

app:layout_constraint<Horizontal/Vertical>_chainStyle: <estilo>

Nota*: *weighted* no es un tipo de cadena. Para lograr ese comportamiento hay que añadir pesos a cada *View* y poner su magnitud correspondiente en *Odp*.

Ejemplo de cadena en ConstraintLayout

```
<androidx.constraintlayout.widget.ConstraintLayout  
... >
```

activity_main.xml

```
<Button
```

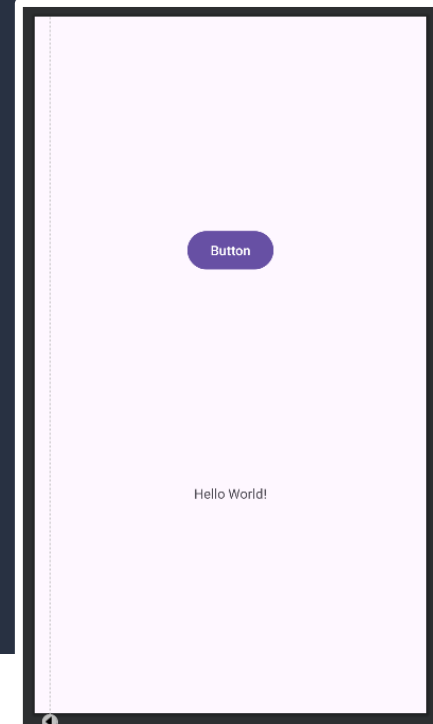
```
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    app:layout_constraintBottom_toTopOf="@+id/textView"  
    app:layout_constraintEnd_toEndOf="parent"  
    app:layout_constraintStart_toStartOf="parent"  
    app:layout_constraintTop_toTopOf="parent"  
    app:layout_constraintVertical_chainStyle="spread" />
```

Restricciones
complementarias
forman una cadena

```
<TextView
```

```
    android:id="@+id/textView"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    app:layout_constraintTop_toBottomOf="@+id/button"  
    app:layout_constraintBottom_toBottomOf="parent"  
    app:layout_constraintEnd_toEndOf="parent"  
    app:layout_constraintStart_toStartOf="parent" />
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```



Ejercicio 1

Diseñar el layout que aparece en la figura de la derecha haciendo uso de los ViewGroup y elementos de la interfaz necesarios.

Recuerda que puedes anidar ConstraintLayout

Atributos de interés:

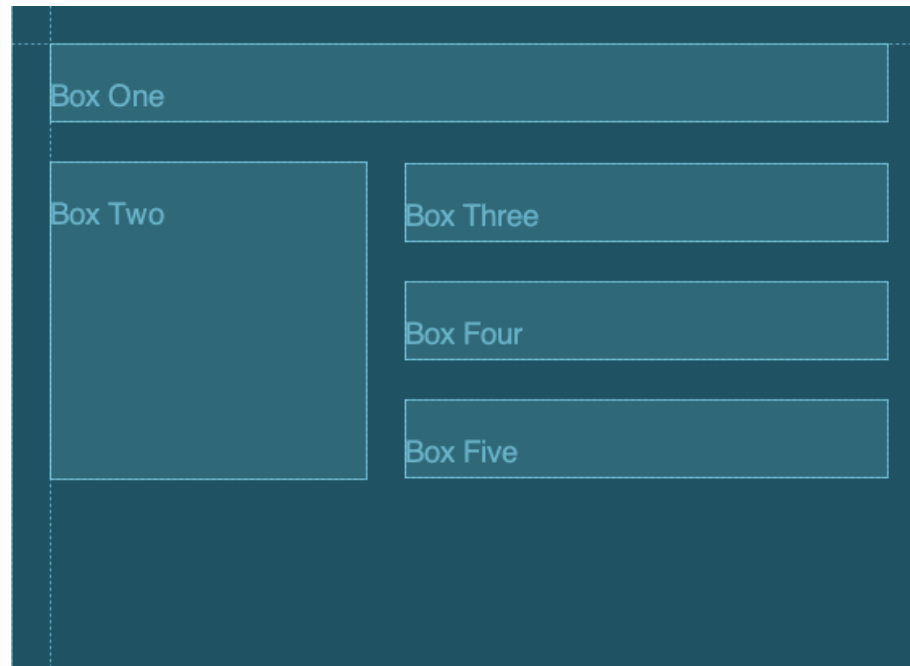
- **android:background** (color del fondo)
- **android:padding(X)**
- **android:layout_margin(X)**
- **app:layout_constraint(Vertical|Horizontal)_weight** (para las cadenas)



Mejorando el diseño

Guías

- Permite posicionar varias vistas en relación con una única guía
- Pueden ser verticales u horizontales
- Permiten una mayor colaboración con los equipos de diseño/UX
- No se dibujan en el dispositivo



Guías

```
<androidx.constraintlayout.widget.Guideline
```

activity_main.xml

```
    android:id="@+id/guideline"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:orientation="vertical"
```

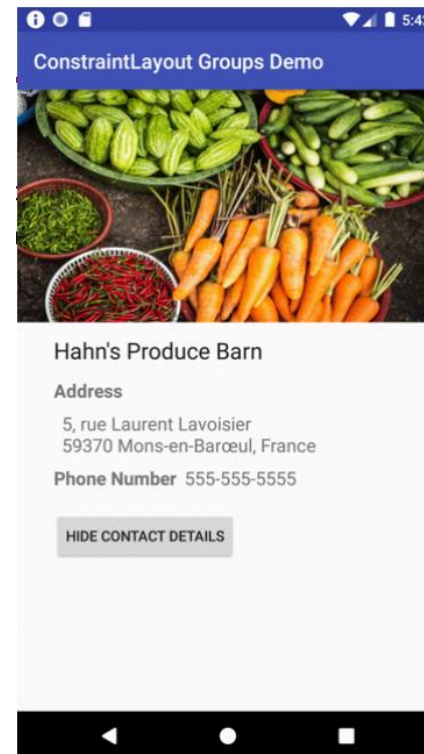
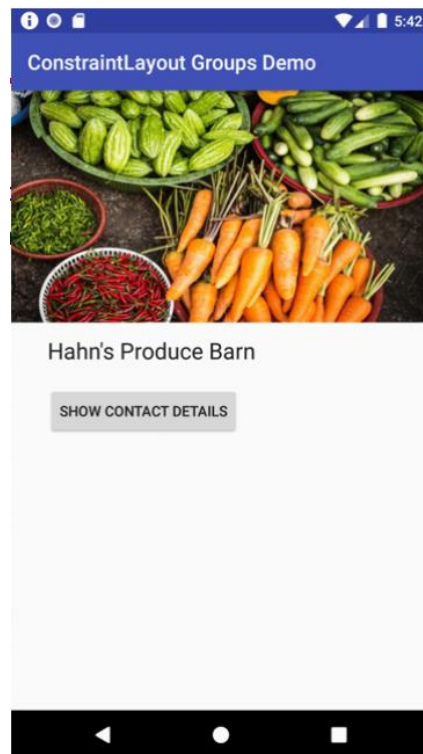
```
    app:layout_constraintGuide_begin="16dp" />
```

Atributos para controlarlas

- `android:orientation`
- `layout_constraintGuide_begin`
- `layout_constraintGuide_end`
- `layout_constraintGuide_percent`

Agrupamiento de elementos

- Los *Group* permite “asociar” elementos de la interfaz creando un agrupamiento
- La visibilidad del agrupamiento se puede controlar y ocultar y mostrar todos los elementos a la vez



Ejemplo de agrupamiento

1. En el XML se crea un Group que asocia diferentes vistas (*referenced_ids*)
2. En el .kt se puede modificar su atributo *visibility* (VISIBLE | GONE)

```
class MainActivity : AppCompatActivity() {  
    private lateinit var grupo: Group  
    private lateinit var boton: Button  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
        boton = findViewById(R.id.boton)  
        grupo = findViewById(R.id.grupo)  
        boton.setOnClickListener {  
            if (grupo.visibility == View.VISIBLE){  
                grupo.visibility = View.GONE  
            }else{  
                grupo.visibility = View.VISIBLE  
            }  
        }  
    }  
}
```

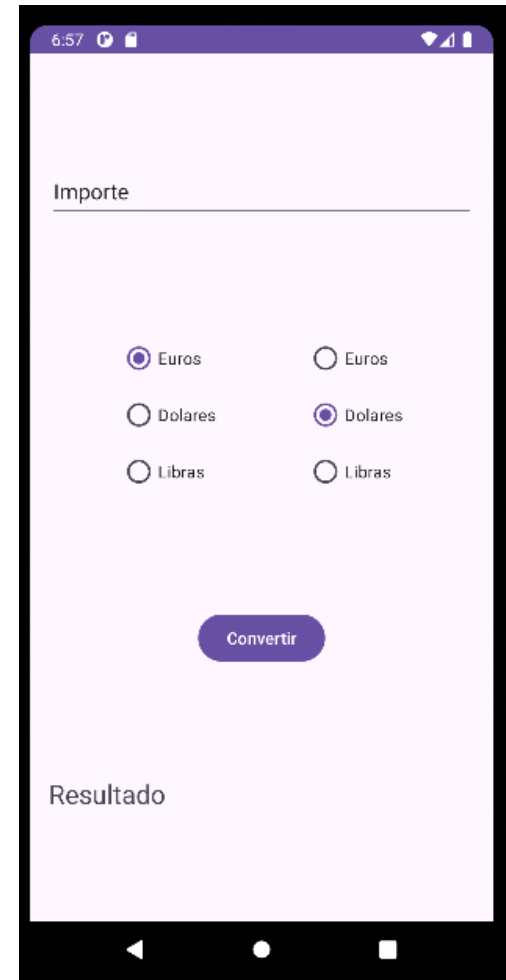
MainActivity.kt

```
<androidx.constraintlayout.widget.Group  
    android:id="@+id/grupo"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    app:constraint_referenced_ids="boton1,textview2"/>
```

activity_main.xml

Ejercicio 2

1. Diseña la aplicación que aparece en la figura de la derecha haciendo uso de los ViewGroup y elementos de la interfaz necesarios.
2. Añade la funcionalidad necesaria para que al pulsar el botón Convertir, se calcule el cambio de divisa para el importe introducido en el campo de texto de arriba teniendo en cuenta la divisa de origen (izquierda) y la de destino (derecha).
3. Modifica los atributos del campo de texto de arriba para permitir al usuario introducir únicamente números decimales.



Analizando nuestra interfaz

Análisis de accesibilidad

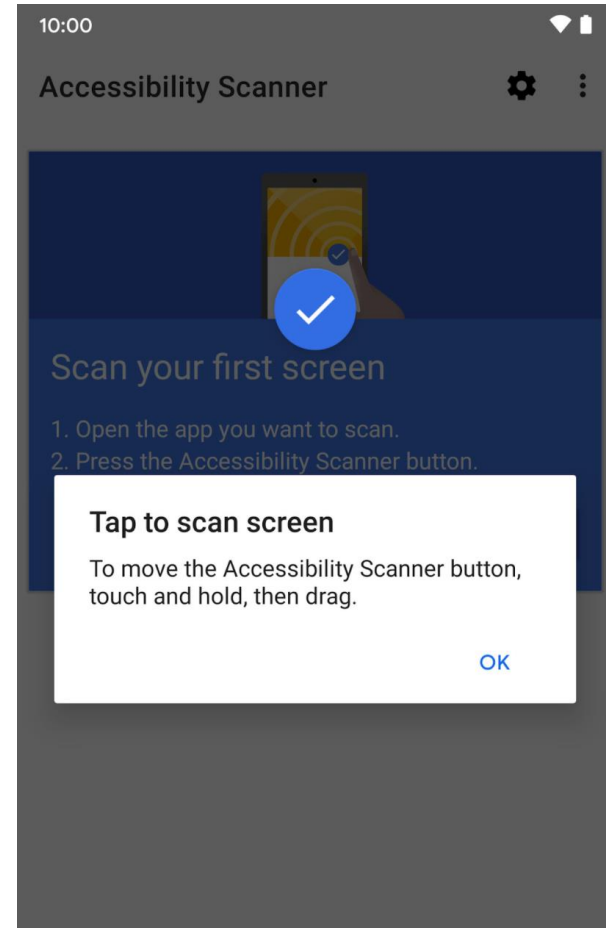
La accesibilidad busca la **mejora del diseño** y la funcionalidad de una aplicación **para facilitar su uso a más personas**, incluidas aquellas con discapacidades.

Hacer una app más accesible mejora la experiencia y **beneficia a todos los usuarios**.

Test de accesibilidad / Accessibility Scanner

App que escanea tu pantalla y sugiere mejoras para hacer tu aplicación más accesible:

- Tamaños de objetivos táctiles
- Vistas en las que se puede hacer clic
- Contraste de texto e imagen



Ejercicio 3

1. Instala la aplicación Test de accesibilidad desde la Play Store
2. Sigue los pasos indicados para habilitar el análisis de la pantalla
3. Abre cualquier aplicación que tengas instalada, pásale el escáner y explora las recomendaciones

Test de Accesibilidad

Google LLC

4,2★
12,4 mil reseñas

1 M+
Descargas

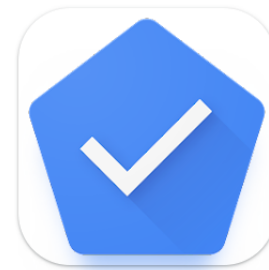
3
PEGI 3

Descargar

Compartir

Añadir a la lista de deseos

Esta aplicación está disponible para todos tus dispositivos



¿Te interesa?

Android tiene más opciones de accesibilidad como *TalkBack* o la navegación por switches

Referencias

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>